Aimaan Ahmed

ID - 210041204

Design Patterns Report

Scenario: Multi-Channel Payment Processing System

Language: Java

---

1. Scenario

> A payment gateway must route transactions through multiple channels (Credit Card, Bank Transfer), apply a pluggable fee calculation, follow a fixed processing workflow, and support transparent add-ons such as audit logging without modifying any existing processor class.

Each requirement maps to exactly one pattern:

Requirement	**Pattern**
Swap fee algorithms at runtime	**Strategy**
Enforce a fixed step-by-step workflow	**Template Method**
Hide which processor class to create	**Factory**
Add logging without touching processors	**Decorator**

---

2. Pattern Roles

Strategy — `FeeStrategy.java`

Intent: Encapsulate interchangeable algorithms behind a common interface.

Class	**Fee Rule**
`FlatFeeStrategy`	Fixed dollar amount regardless of transaction size
`PercentageFeeStrategy`	Rate x amount (e.g. 2.9%)

The strategy is injected into any `PaymentProcessor` at construction time. Swapping the strategy changes the fee math without touching the processor.

Template Method — `PaymentProcessor.java`

Intent: Define the skeleton of an algorithm in a base class; defer specific steps to subclasses.

`PaymentProcessor.process()` enforces this exact sequence for every payment:

```
1. validate(payer, amount)    -- abstract: each channel validates differently
2. calculateFee via Strategy  -- concrete: always delegated to FeeStrategy
3. charge(payer, amount, fee) -- abstract: each channel charges differently
4. notify(payer)              -- abstract: each channel notifies differently
```

Concrete subclasses (`CreditCardProcessor`, `BankTransferProcessor`) only implement the three abstract hooks; the sequence itself is never duplicated.

Factory — `PaymentFactory.java`

Intent: Centralise object creation so clients never call `new` on a concrete class.

```
PaymentFactory.create("credit_card")    // new CreditCardProcessor(new
PercentageFeeStrategy(2.9))
PaymentFactory.create("bank_transfer")  // new BankTransferProcessor(new
FlatFeeStrategy(2.50))
```

The Factory also wires the default Strategy into each processor, binding both patterns at a single, controlled creation site.

Decorator — `PaymentProcessorDecorator.java`

Intent: Add behaviour to an object dynamically by wrapping it in an object that shares the same interface.

`PaymentProcessorDecorator` extends `PaymentProcessor` and holds a `wrapped` reference. `LoggingDecorator` overrides `process()` to print before/after messages, then delegates to `wrapped.process()`.

```
LoggingDecorator
  |-- CreditCardProcessor   (any processor works, transparently)
```

The processor never knows it is wrapped; the caller never changes its code.

---

3. How the Patterns Interact

```
Main
 |
 |--[Factory]--> PaymentFactory.create("credit_card")
 |                   |-- new CreditCardProcessor( new PercentageFeeStrategy(2.9)
)
 |                               |                          |
 |                          [Template]               [Strategy]
 |
 |--[Decorator]--> new LoggingDecorator( creditCardProcessor )
 |
```

```
 +-- loggedProcessor.process("Alice", 200.00)
         |
         |-- [Decorator] prints LOG START
         |-- [Template]  validate -> fee(Strategy) -> charge -> notify
         +-- [Decorator] prints LOG END
```

---

4. Verified Output

```
=== Demo 1: Credit Card (2.9% fee) + Logging ===
>> [LOG] START payment | payer=Alice | amount=$200.0 | channel=CreditCard

[CreditCard] Processing payment for Alice
  Validating card for: Alice ... OK
  Fee (2.9% of amount): $5.80
  Charged $200.00 + $5.80 fee to card.
  Receipt emailed to Alice
>> [LOG] END payment for Alice -- done.

=== Demo 2: Bank Transfer (flat $2.50 fee) + Logging ===
>> [LOG] START payment | payer=Bob | amount=$500.0 | channel=BankTransfer

[BankTransfer] Processing payment for Bob
  Validating bank account for: Bob ... OK
  Fee (Flat $2.5): $2.50
  ACH transfer of $500.00 + $2.50 fee initiated.
  Confirmation SMS sent to Bob
>> [LOG] END payment for Bob -- done.

=== Demo 3: Credit Card with custom Flat $1 fee (no logger) ===

[CreditCard] Processing payment for Carol
  Validating card for: Carol ... OK
  Fee (Flat $1.0): $1.00
  Charged $350.00 + $1.00 fee to card.
  Receipt emailed to Carol
```

---

5. File Reference

File	Pattern	Contents
`FeeStrategy.java`	Strategy	Interface + FlatFee + PercentageFee
`PaymentProcessor.java`	Template Method	Abstract base + CreditCard + BankTransfer
`PaymentProcessorDecorator.java`	Decorator	Abstract base + LoggingDecorator
`PaymentFactory.java`	Factory	Static factory
`Main.java`	Demo	3 scenarios